

EverGreen Cold Storage

Demo & Verification Pack

Combined report of:

1. Flow Verification Report
2. Demo Guide
3. Use Cases

Overall result: PASS — ready for demo

Date: 7 October 2026

Merchant: info@evergreen.com

Login password: Evergreen@123

Contents

- Part 1 — Flow Verification Report
- Part 2 — Demo Guide (10–15 minute walkthrough)
- Part 3 — Use Cases (UC-01 to UC-12)

Part 1 — Flow Verification Report

Date: 7 Oct 2026

Merchant checked: info@evergreen.com (EverGreen Cold Storage)

Overall result: **PASS — ready for demo**

How this was checked

- Automated feature tests (tests/Feature/ColdStorage/)
- Demo data seeder (ColdStorageDemoFlowSeeder)
- Live DB health command (php artisan cold-storage:verify-flow)

Automated tests — 19 / 19 passed (72 assertions)

Area	Result	What was proven
Full lifecycle	PASS	Receipt → dispatch → transfer → damage → bill → payment → temperature → reservation
Partial dispatch	PASS	Remaining stock stays with customer
Over-dispatch	PASS	Quantity above available is rejected
Double withdrawal	PASS	Cannot take stock already dispatched
Wrong customer	PASS	Cannot withdraw another customer's lot
Wrong branch chamber	PASS	Chamber from another branch is blocked
Transfer	PASS	Location changes; customer total unchanged
Damage	PASS	Reason required; stock reduces
Cancel dispatch	PASS	Stock reverses correctly
Billing overlap / rates	PASS	Posted bills stay fixed; overlapping periods blocked
Billing quantity rules	PASS	Remaining qty + arrival-day rules work
Service charges	PASS	Payments do not create shop sales stock
Temperature alerts	PASS	Out-of-range flagged; can acknowledge
Occupancy / staff branch	PASS	Capacity math + branch scope
Lot ageing	PASS	Days-in-store report
Bill from stock	PASS	Draft lines built from remaining stock
Alert email digest	PASS	Dedicated recipients; skips when empty

Live database verify-flow — 13 / 13 passed

Check	Status	Detail
Active chambers	PASS	3
Posted goods receipts	PASS	3
Posted dispatches	PASS	3
Stock movements	PASS	6
Stock on hand (packages)	PASS	40.00
Posted storage bills	PASS	2 / due 2,800.00
Temperature readings	PASS	3

Check	Status	Detail
Reservations confirmed	PASS	2
CS isolation from sales	PASS	Confirmed in tests
Stock statement report	PASS	2 rows
Capacity heatmap	PASS	3 rows
Lot ageing report	PASS	2 rows
Occupancy service	PASS	Working

Demo data created

After running ColdStorageDemoFlowSeeder:

- Customer: **Ahmed Traders**
- Chamber: **Demo Chamber A** (+ Demo Rack A)
- Posted goods receipt with lot **LOT-DEMO-1** (40 bags in, 10 dispatched → 30 remaining)
- Posted storage bill (where rates allow)
- Out-of-range temperature reading (for Action alerts)
- Confirmed reservation for next week

Login for demo: info@evergreen.com / Evergreen@123

Known limits (not failures)

Item	Notes
Alert emails	Turned off in config/ui-modules.php (cold_storage_alert_emails = false). Alerts still show in the panel.
Reservations vs occupancy	Confirmed reservations do not yet reduce free capacity on the heatmap. Occupancy is physical stock only.
Legacy shop modules	Hidden via UI modules for cold-storage-first demos; code still exists.

Re-run anytime

```
php artisan db:seed --class=ColdStorageDemoFlowSeeder
php artisan cold-storage:verify-flow
php artisan test tests/Feature/ColdStorage/
```

Part 2 — Demo Guide

Easy walkthrough for showing the product to a customer (about 10–15 minutes).

Login

- **URL:** /merchant/login
- **Email:** info@evergreen.com
- **Password:** Evergreen@123

What this software is

EverGreen Cold Storage is a **merchant panel for cold stores**.

Customers keep their own goods with you (bags, crates, weight). You:

- Receive goods into a chamber
- Track where they sit
- Dispatch (hand back) when the customer asks
- Charge storage rent and collect payment

It is **not** a normal shop POS. Goods stay customer-owned until they leave.

Demo story (10–15 minutes)

1) Home dashboard

Open **Dashboard**. Show: Receiving / Dispatch packages, Stock on hand, Billing due, Chamber occupancy, Action alerts / lot ageing shortcuts.

Tip: set **Date From / Date To** to cover the demo bill period if Billing shows zero.

2) Setup (if asked how do we start?)

Under Cold Storage / setup menus: Business + Branch, Chamber (capacity + temperature limits), optional rack/bin locations, Customer, Product (e.g. Potatoes), Storage rate card.

3) Goods receipt

Goods receipts → **Create** — Branch + customer + date → add lot lines → allocate chamber/rack → Save → Post. After post, stock appears on the dashboard.

4) Temperature check

Temperature readings → **Create** — Enter a value outside chamber min/max. It shows as out of range → Action alerts → Acknowledge after checking.

5) Dispatch (customer takes goods)

Goods dispatches → **Create** — Same customer, pick lots still in stock, quantity cannot exceed available, then Post. Stock drops. No sale invoice is created.

6) Damage / adjustment (optional)

Damage and adjustments — Record torn bags with a reason and negative quantity. Post it.

7) Storage bill

Storage bills — Use Bill from stock (fast) or Create manually. Choose customer, period, charge basis → Calculation preview → Post → record payment. Dashboard Billing card updates for that period.

8) Extra modules to flash

- **Lot ageing** — oldest lots still in store
- **Capacity heatmap** — how full each chamber is

- **Reservations** — book space before goods arrive (Confirm / Fulfill / Cancel)

Reset / refresh demo data

```
php artisan db:seed --class=ColdStorageDemoProductsSeeder
php artisan db:seed --class=ColdStorageDemoFlowSeeder
php artisan cold-storage:verify-flow
```

What not to demo as sold stock

Cold storage movements never create classic **Sales / Purchases** product stock.

If someone asks: Is this my shop inventory? → **No — this is customer-owned storage.**

Part 3 — Use Cases

Plain-language situations the system is built for.

UC-01 — New customer stores goods

Actor: Store operator

Goal: Put customer bags into a chamber and know the lot number.

Steps: Create customer → create goods receipt → allocate chamber → post receipt.

Result: Stock on hand increases; lot can be found later for dispatch or billing.

UC-02 — Customer takes part of their stock

Actor: Store operator

Goal: Hand over only some bags from a lot.

Steps: Create dispatch → choose same customer + lot → enter quantity ≤ available → post.

Result: Remaining stock stays; over-withdrawal is blocked.

UC-03 — Wrong branch or wrong customer blocked

Actor: System

Goal: Stop mistakes that mix stores or owners.

Steps: Chamber from another branch on a receipt; dispatch for a different customer than the lot owner.

Result: Post fails with a clear error.

UC-04 — Move bags inside the store

Actor: Store operator

Goal: Shift stock from Rack A to Rack B without changing customer total.

Steps: Create transfer → from/to locations → post.

Result: Location balances change; customer still owns the same total packages.

UC-05 — Record damaged goods

Actor: Store operator

Goal: Reduce stock when bags are torn or spoiled.

Steps: Damage adjustment with a written reason → negative quantity → post.

Result: Stock drops; reason is kept for audit.

UC-06 — Charge storage rent

Actor: Billing clerk

Goal: Bill a customer for days goods stayed in store.

Steps: Ensure rate card exists → create bill (or Bill from stock) for a date period → preview → post → take payment.

Result: Due amount shows on dashboard; payments reduce balance. Overlapping posted periods for the same lot are rejected.

UC-07 — Watch chamber temperature

Actor: Operator / supervisor

Goal: Know when a chamber is outside safe limits.

Steps: Record temperature → system compares to chamber min/max → Action alerts list open exceptions → acknowledge after checking.

Result: Exceptions are visible until acknowledged.

UC-08 — See how old stock is

Actor: Supervisor

Goal: Find lots sitting too long.

Steps: Open Lot ageing.

Result: Lots sorted by days in store.

UC-09 — See chamber fullness

Actor: Supervisor

Goal: Know which chambers are nearly full.

Steps: Open Capacity heatmap.

Result: Fill % per chamber (low → critical).

UC-10 — Book space before arrival

Actor: Operator

Goal: Reserve chamber space for a future delivery.

Steps: Create reservation → Confirm → later Fulfill (link receipt) or Cancel with reason.

Result: Upcoming bookings appear on Action alerts (next 7 days).

UC-11 — Undo a wrong dispatch

Actor: Supervisor

Goal: Reverse a posted withdrawal.

Steps: Open posted dispatch → Cancel with reason.

Result: Stock returns; document marked cancelled.

UC-12 — Keep cold storage separate from shop sales

Actor: System

Goal: Customer-owned goods must not look like shop inventory sold.

Steps: Post receipts / dispatches / bills as cold-storage documents only.

Result: Posting does not create Sale or Purchase stock documents.

End of report

EverGreen Cold Storage — ready for demo